

A Decoder with Reinforcement Learning Feedback

Milad Taghipour, Bane Vasić, *Fellow, IEEE*

Department of Electrical and Computer Engineering, the University of Arizona, Tucson, AZ

Email: miladt@arizona.edu, vasic@ece.arizona.edu

Abstract—This paper explores the application of reinforcement learning techniques to improve the performance of bit-flipping decoders and finding their optimal decisions. We begin by providing an overview of bit-flipping based decoders and reinforcement learning algorithms. We then outline the methodology for mapping the iterative decoding process into Markov Decision Processes (MDPs). We propose a feedback-based method to exploit and enhance the performance of existing well-established decoders by applying reinforcement learning algorithms after a selected decoder. In essence, we modify the MDP to reduce the number of states, making reinforcement learning algorithms feasible for low-rate and long-length codes. In addition, focusing on the correction of dominant error patterns of a selected decoder improves its error correction capability. Finally, we present experimental results for the Binary Symmetric Channel (BSC) to demonstrate the efficiency of the proposed methods.

Index Terms—Reinforcement Learning, bit-flipping algorithm, Iterative Decoders, trapping sets, linear Block codes, LDPC codes, Error-floor.

I. INTRODUCTION

ERROR correcting codes are critical for reliable communication over noisy channels. While LDPC codes and iterative decoders can asymptotically approach channel capacity, their performance in the finite-length regime remains suboptimal. This limitation is primarily due to trapping sets [1], [2] and pseudocodewords [3], which often have lower weight than the minimum distance of the code [4], leading to error floors at high SNR. These structures hinder iterative decoders from achieving maximum likelihood performance. To mitigate this, several approaches have been proposed, including modifications to belief propagation [5]–[8] and the use of Finite Alphabet Iterative Decoders (FAIDs) [9], [10]. Recent advances in artificial intelligence have motivated interest in designing efficient decoders for finite-length codes. Developing efficient, high-performance decoders with low computational complexity remains challenging for short to medium-length codes. One approach involves parameterized belief propagation (BP) decoders [11]–[14], which retain the advantages of standard BP while improving convergence speed and error-correction performance. The authors in [15] presented the generalization gap of the Neural BP decoder. An alternative approach is to view the problem of decoding as a decision-making problem,

using reinforcement learning (RL) to learn optimal policies. In this context, decoding is modeled as a Markov Decision Process (MDP), where RL guides sequential bit-flipping decisions [16], [17]. Similarly, [18] formulates belief propagation scheduling as an MDP, optimizing check-node scheduling to improve performance over conventional flooding strategies. In this paper, we propose a feedback decoding strategy designed to further enhance the performance of well-established iterative decoders in the literature. This approach leverages an RL block to address error patterns that a decoder fails to correct, thereby effectively expanding its correctable region. In general, this method can be employed to improve state-of-the-art decoders and strengthen their error correction capabilities. The remainder of this paper is organized as follows. Section II provides the necessary background on channel coding. Section III introduces the MDP framework and RL-based bit-flipping strategy. In Section IV, we propose the RL feedback decoding technique to improve decoder performance. The numerical results are presented in Section V. The paper concludes with a summary of the findings in Section VI.

II. CHANNEL CODING BACKGROUND

Let us assume $\mathcal{C}$ to be a binary linear code with parameters $(n, k, d_{\min})$, where n is the code length, k is the code dimension, and $d_{\min}$ is the minimum distance of the code. We define a binary linear code $\mathcal{C} = \{\mathbf{c} \in \mathbb{F}_2^n \mid \mathbf{c}H^T = \mathbf{0}\}$, where H is the parity-check matrix, which is an $m \times n$ binary matrix. We also define $\mathbf{h}_{c_i}$ as the i -th column of the parity check matrix H . The error correction capability of a code can be defined on the basis of its minimum distance as $t = \lfloor (d_{\min} - 1)/2 \rfloor$. A codeword $\mathbf{c}$ is transmitted through the Binary Symmetric Channel (BSC) as $\mathbf{y} = (1 - 2\mathbf{c}) + \mathbf{z}$, where $\mathbf{y}$ is the received row vector of length n , and $\mathbf{z}$ is a row vector of length n generated by i.i.d. Bernoulli random variable with parameter ρ which is the cross-over probability of the channel.

Let $\mathbf{y} = (y_1, y_2, \dots, y_n)$ denote the input provided to the decoder, and let $\mathbf{x}^\ell = (x_1^\ell, x_2^\ell, \dots, x_n^\ell)$ for $\ell \leq D$ represent the output vector at the ℓ -th iteration. The decoder operates until it either finds a valid codeword or reaches the predefined maximum number of iterations D , whichever comes first. As a result, the decoder's final output will be either a valid codeword or $\mathbf{x}^D$. The bit-flipping algorithm is a fast, simple yet effective algorithm in the decoding of error correcting codes such as LDPC codes. The idea is to flip bits that are the least reliable to maximize the number of satisfied checks [19].

The authors acknowledge the support of the National Science Foundation under grants CIF-2420424, CIF-2106189, CCF-2100013, CCSS-2052751, and CCSS-2027844. Bane Vasić has disclosed an outside interest in his company Codelucida to The University of Arizona. Conflicts of interest resulting from this interest are being managed by The University of Arizona in accordance with its policies.

The BF algorithm with majority update rule is provided in Algorithm 1, where $\mathbf{s}$ is a syndrome row vector of length m , $\mathbf{q} = (q_1, \dots, q_n)$ is an n -length row vector representing the number of unsatisfied checks connected to each variable, $\mathcal{B}$ is the set of candidates for flipping based on the majority update rule, v_i is the i -th variable node, and $\oplus$ denotes modulo-2 addition.

Algorithm 1 Bit-Flipping Decoder with Majority Update Rule

Input: $\mathbf{x} = \text{sign}(\mathbf{y})$, H (parity-check matrix), maximum iteration number.

Output: estimated codeword $\hat{\mathbf{c}}$.

```

1:  $\hat{\mathbf{c}} = \mathbf{x}$ 
2:  $\mathbf{s} = \text{mod}(\hat{\mathbf{c}}H^T, 2)$  ▷ Syndrome Computation
3: while  $\mathbf{s} \neq 0$  and maximum iterations is not reached do
4:    $\mathbf{q} = \mathbf{s}H$  ▷ Compute violated parity checks
5:    $\mathcal{B} = \{i \in [n] \mid q_i \geq \lfloor \frac{\deg(v_i)}{2} \rfloor + 1\}$  ▷ Update Rule
6:    $\hat{c}_i \leftarrow \hat{c}_i \oplus 1; \quad \forall i \in \mathcal{B}$  ▷ Flip Bit
7:    $\mathbf{s} = \text{mod}(\hat{\mathbf{c}}H^T, 2)$ 
8: end while

```

III. MARKOV DECISION PROCESS AND BIT-FLIPPING STRATEGY

MDPs provides a mathematical framework for modeling decision-making problems where outcomes are random or deterministic under the control of a decision maker [20]. An MDP can be defined based on (S, A, P, R, γ) , where S is the set of finite states representing all situations of the environment, A is the set of finite actions that the decision maker, or agent, can take, $P_{ss'}^a = P(s' \mid s, a) \triangleq P(S_{t+1} = s' \mid S_t = s, A_t = a)$ is the probability transition function $P : S \times A \times S \rightarrow [0, 1]$ which represents the probability of going to state s' from state s after taking action a , $R(s, a)$ is the reward function $R : S \times A \rightarrow \mathbb{R}$ which represents the immediate reward received after taking action a in state s , and $\gamma \in [0, 1]$ is the discount factor which denotes the importance of future rewards. A deterministic policy π is a mapping $\pi : S \rightarrow A$, where $\pi(s)$ denotes the action to take in state s . The goal in an MDP is to find an optimal policy π^* that maximizes the expected cumulative reward $\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi}[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t)) \mid s_0 = s]$, where $\mathbb{E}_{\pi}$ is the expectation by taking actions under policy π . In the context of the BF strategy, we define the state and action spaces, as well as the reward function, following the formulation in [16], as detailed below.

A. State and Action Spaces

Given that the code dimension is n , each of these n bits can be erroneous. In other words, the action space A should be designed to allow the flipping of each bit at any time step. Therefore, the set of actions is $A = \{1, 2, \dots, n\}$. The decoding process can be terminated based on the value of the syndrome vector, which indicates the positions of unsatisfied checks in the code. Consequently, the state space S is defined by all possible binary syndromes of length m . The initial state

$s_0 = \mathbf{y}H^T$ represents the syndrome of the received vector. Next, we need to define the transition probability function $P(s' \mid s, a)$ for an MDP. By taking action a from state s , we transition to state $s' = s + \mathbf{h}_{c_a}$ with probability 1, indicating that it is a deterministic MDP. Terminal state corresponds to the all-zero syndrome, indicating that a codeword is obtained and that the decoding process should be terminated. We also impose a limit of L steps for decoding.

B. Reward Strategy

Choosing a reward strategy is crucial in RL algorithms because it significantly impacts the learning efficiency and the resulting policy. Since the goal of the decoding problem primarily relies on finding a codeword, we define a positive reward of 1 if an agent reaches the terminal state. It is also necessary to penalize the action of flipping a bit at each step in order to encourage the decoder to find a solution that requires the fewest possible bit flips. This helps identifying the set of least reliable bits to find a codeword, thereby minimizing the probability of word error rate. Thus, we define a reward function for the BSC

$$R(s, a, s') = \begin{cases} -\frac{1}{L} + 1, & \text{if } s' = 0 \\ -\frac{1}{L}, & \text{otherwise.} \end{cases} \quad (1)$$

C. Q-learning Algorithm

Q-learning is a model-free reinforcement learning algorithm that is used to solve MDPs. It has the ability to learn the optimal policy π^* directly from interactions with the environment without requiring the model of environment. The action-value function $Q : S \times A \rightarrow \mathbb{R}$ is defined to be the expected cumulative rewards starting from state s , taking action a , and using policy π to select actions afterwards

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t)) \mid S_0 = s, A_0 = a]. \quad (2)$$

Q-learning algorithm aims to learn the optimal action-value function

$$Q^*(s, a) = \max_{\pi} Q_{\pi}(s, a) \quad (3)$$

that find the expected reward of taking action a in state s and following the optimal policy afterwards. The Q-learning algorithm iteratively updates the Q-values using the following rule

$$Q(s, a) \leftarrow Q(s, a) + \alpha[R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (4)$$

updating the Q-values based on the immediate rewards received and the estimated value of subsequent maximum action-value estimate, where $\alpha \in (0, 1]$ is the step size for updating the Q values. The Q-learning algorithm is given in Algorithm 2 [20], where s_E is the set of terminal states. It is worth noting that the complexity of Q-learning algorithm is $O(|S| \times |A|^2)$, due to the need to update each state-action pair and compute the maximum Q-value over all possible actions in the subsequent

Algorithm 2 Q-learning Algorithm

Input: Initialize $Q(s, a)$ arbitrarily (e.g., randomly), except $Q(s_e, a) = 0, \forall s_e \in S_E$. Set the number of episodes N_{episodes} .

Output: Optimal action-value function $Q(s, a)$ for all $s \in S, a \in A$.

- 1: **for** $i \leftarrow 1$ to N_{episodes} **do**
- 2: Initialize state s
- 3: **while** $s \notin S_E$ **do**
- 4: Choose action a from state s using an exploration-exploitation strategy (e.g., ϵ -greedy)
- 5: Take action a , observe reward $R(s, a)$ and next state s'
- 6: Update Q -value:

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha \left[R(s, a) + \gamma \max_{a'} Q(s', a') \right]$$
- 7: $s \leftarrow s'$
- 8: **end while**
- 9: **end for**

state during each iteration, as described in Equation (4). We employ the ϵ -greedy strategy for the action selection step.

D. Deep Q-learning Algorithm

Deep Q-learning is an extension of Q-learning, a popular RL algorithm, that uses deep neural networks to approximate the Q-function [21]. Q-learning stores a real-valued table of size $|S| \times |A|$, which is infeasible for low-rate or long codes. We assume $Q_\theta(s, a)$ represents the estimation of the Q-function with learnable parameters θ . We store transition tuples (s, a, r, s') obtained from simulating the MDP in a set $\mathcal{B}$ called replay memory. For stable training, we use two neural networks a primary Q-network $Q_\theta^p(s, a)$ for selecting actions and a target Q-network $Q_\theta^t(s, a)$ to provide more stable Q-value updates. The agent balances exploration and exploitation using an ϵ -greedy policy with linear decay. Then, we update the parameter θ based on reducing the loss function

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \in \mathcal{B}} [r + \gamma \max_{a' \in A} Q_{\theta^-}^t(s', a') - Q_\theta^p(s, a)]^2 \quad (5)$$

where gradient descent based methods are employed to update the parameters θ to minimize the loss function defined in Equation (5). Periodically, after some iterations the target network is updated to match the primary network $\theta^- \leftarrow \theta$, improving the stability of training. The goal is to maximize cumulative rewards by learning an optimal policy over time. The full algorithm is shown in Algorithm 3.

IV. FEEDBACK DECODER

As explained in Section III-C, the complexity of the Q-learning update rule is $O(|S| \times |A|^2)$, which is proportional to the number of states and the square of the number of actions. For the BF MDP defined in Section III-A, this becomes $O(n^2 2^{n-k})$. In this section, we aim to establish a general framework for reducing the number of states in the MDP. Due

Algorithm 3 Deep Q-learning Algorithm

Input: Initialize replay memory $\mathcal{B}$, initialize Q-networks $Q_\theta^p(s, a)$ and $Q_\theta^t(s, a)$ with random weights θ and θ^- , and set epo as the number of episodes.

Output: Primary Q-network $Q_\theta^p(s, a)$

- 1: **for** $i = 1, 2, \dots, \text{epo}$ **do**
- 2: Sample a random minibatch (s, a, r, s') of transitions from $\mathcal{B}$.
- 3: Set $y \leftarrow \begin{cases} r, & \text{if } s' \in S_E \\ r + \gamma \max_{a'} Q_\theta^t(s', a'), & \text{otherwise} \end{cases}$
- 4: Perform a gradient descent step on $(y - Q_\theta^p(s, a))^2$ with respect to θ .
- 5: **if** fixed number of steps are reached **then**
- 6: Update $Q_\theta^t \leftarrow Q_\theta^p$
- 7: **end if**
- 8: **end for**

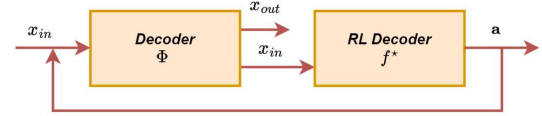


Fig. 1: Feedback based Reinforcement Learning decoder.

to the large state and action spaces associated with low-rate and long-length codes, this approach becomes computationally infeasible. Thus, we propose a general feedback method to leverage the advantages of well-established codes and decoders from the literature. We incorporate an RL block to correct errors that the selected decoder fails to correct, thereby enhancing its performance and expanding its overall correctable region through an RL-based mechanism. In addition, the RL-based feedback decoder can be applied iteratively up to L times to further improve performance, as illustrated in Figure 1. We define $\mathcal{R}_c^\Phi$ to be the successful decoding region of a chosen decoder Φ

$$\mathcal{R}_c^\Phi = \{x \in \mathbb{F}_2^n \mid \Phi(x) = \arg \min_{c' \in \mathcal{C}} d(x, c')\}. \quad (6)$$

We also define the miscorrectable region of the decoder Φ as the set of channel vectors for which the decoder assigns a codeword to the received word that is not the closest codeword.

$$\mathcal{R}_m^\Phi = \{x \in \mathbb{F}_2^n \mid \Phi(x) \in \mathcal{C}, \Phi(x) \neq \arg \min_{c' \in \mathcal{C}} d(x, c')\} \quad (7)$$

Then, we can define the set of failure-inducing error patterns as the complement of the set of correctable and miscorrectable error patterns

$$\mathcal{R}_f^\Phi = \{x \in \mathbb{F}_2^n \mid \Phi(x) \notin \mathcal{R}_c^\Phi \cup \mathcal{R}_m^\Phi\}. \quad (8)$$

We denote by S_i^Φ the set of syndromes corresponding to the i (correctable, failure, or miscorrectable) decoding region $\mathcal{R}_i^\Phi$ of the decoder Φ , where $S_i^\Phi = \{yH^T \mid y \in \mathcal{R}_i^\Phi\}$ and $i \in$

$\{c, f, m\}$. The state space of the new MDP is $S_f^\Phi \cup \{\mathbf{0}\}$, and the action space is $A = \{1, 2, \dots, n\}$. The reward function is defined as follows

$$R(s, a, s') = \begin{cases} -\frac{1}{L} + 1, & \text{if } s' \notin S_f^\Phi \\ -\frac{1}{L}, & \text{if } s' \in S_f^\Phi. \end{cases} \quad (9)$$

By employing the feedback strategy, the state space of the MDP is reduced from 2^{n-k} to $|S_f^\Phi| + 1$, where $|S_f^\Phi|$ denotes the number of syndromes that the selected decoder Φ fails to correct. The RL block interacts with the existing decoder to expand its correctable region. The general decoding procedure during inference is outlined in Algorithm 4, where $\pi^*(s)$ denotes the optimal policy. Given the convergence of the Q-learning algorithm, we formulate the following theorem to characterize the error-correction capability of the optimal policy.

Algorithm 4 Decoder with Reinforcement Learning Feedback

Input: $\mathbf{x} = \text{sign}(\mathbf{y})$, H (parity-check matrix), and maximum iteration number.

Output: estimated codeword $\hat{\mathbf{c}}$.

```

1:  $\mathbf{x}_{\text{in}} = (1 - \mathbf{x})/2$ 
2:  $\mathbf{s}_{\text{in}} = \text{mod}(\mathbf{x}_{\text{in}} H^T, 2)$ 
3: while  $\mathbf{s}_{\text{in}} \neq \mathbf{0}$  and maximum iterations is not reached do
4:    $\mathbf{x}_{\text{out}} = \Phi(\mathbf{x}_{\text{in}})$   $\triangleright$  output of the chosen decoder
5:    $\mathbf{s}_{\text{out}} = \text{mod}(\mathbf{x}_{\text{out}} H^T, 2)$ 
6:   if  $\mathbf{s}_{\text{out}} \neq \mathbf{0}$  then
7:      $\mathbf{a} = \pi^*(\mathbf{s}_{\text{in}})$   $\triangleright$  Output of RL Block
8:      $\mathbf{x}_{\text{in}} \leftarrow \mathbf{x}_{\text{in}} \oplus \mathbf{a}$ 
9:      $\mathbf{s}_{\text{in}} = \text{mod}(\mathbf{x}_{\text{in}} H^T, 2)$ 
10:     $\hat{\mathbf{c}} = \mathbf{x}_{\text{in}}$ 
11:   else
12:      $\hat{\mathbf{c}} = \mathbf{x}_{\text{out}}$ 
13:     Break
14:   end if
15: end while
```

Theorem 1. *The error correction capability of the proposed feedback decoder, utilizing the reward function defined in Equation (9), is lower bounded by*

$$\left\lfloor \frac{\min_{y \in \mathcal{R}_f^\Phi} w_H(y) + \min_{y \in \mathcal{R}_m^\Phi} w_H(y) - 1}{2} \right\rfloor \quad (10)$$

where $w_H(\cdot)$ denotes the Hamming weight of an error.

Proof. The details of the proof are given in the extended version of this paper in [22]. $\square$

Remark 1. *To correct the entire failure region, it is essential to have precise knowledge of the miscorrection region. Accordingly, the reward function is modified as follows*

$$R(s, a, s') = \begin{cases} -\frac{1}{L} + 1, & \text{if } s' = S_c^\Phi \\ -\frac{1}{L}, & \text{if } s' \in S_f^\Phi \\ -\frac{1}{L} - 1, & \text{if } s' \in S_m^\Phi \end{cases} \quad (11)$$

Then, the error-correction capability of the integrated decoder, after the convergence of training, is lower bounded by the minimum of the weight of the miscorrected error and the error-correction capability of the code, i.e.,

$$\min \left(t, \min_{y \in \mathcal{R}_m^\Phi} w_H(y) \right). \quad (12)$$

To further reduce the number of states, we employ the concept of truncated MDP to decrease the state space size in the following.

A. Feedback Decoder with Truncated MDP

To further reduce the number of states in the MDP of the feedback decoder, we propose the concept of a truncated MDP, where the state space is restricted to states lying within a Hamming ball centered around a codeword. We define a Hamming ball $\mathcal{B}(c, w)$ with a radius w for a binary linear code of length n to cover all received words within a maximum hamming distance of w or less around a codeword $c \in \mathcal{C}$ defined as

$$\mathcal{B}(c, w) = \{\mathbf{x} \in \mathbb{F}_2^n \mid d_H(\mathbf{x}, c) \leq w\} \quad (13)$$

where $d_H(\mathbf{x}, c)$ represents the hamming distance between the codeword c and the received word $\mathbf{x}$. We define a hamming ball $\mathcal{B}(w)$ with radius w of a code $\mathcal{C}$ to be the union of hamming balls centered at each codeword $c \in \mathcal{C}$, i.e., $\mathcal{B}(w) = \cup_{c \in \mathcal{C}} \mathcal{B}(c, w)$. The hamming ball $\mathcal{B}(w)$ includes all received words within a maximum hamming distance of w from any codeword $c \in \mathcal{C}$. $S(w)$ denotes the set of syndromes associated with $\mathcal{B}(w)$, where $S(w) = \{\mathbf{y} H^T \mid \mathbf{y} \in \mathcal{B}(w)\}$. By combining the concept of the feedback decoder with the truncated MDP, the number of states can be further reduced. We define $\mathcal{BR}_c^\Phi(w)$ as the bounded successful decoding region of a given decoder Φ with radius w , represented as

$$\mathcal{BR}_c^\Phi(w) = \mathcal{B}(w) \cap \mathcal{R}_c^\Phi. \quad (14)$$

Similarly, we define the bounded failure and miscorrection decoding regions of the decoder Φ as

$$\mathcal{BR}_f^\Phi(w) = \mathcal{B}(w) \cap \mathcal{R}_f^\Phi \quad (15)$$

$$\mathcal{BR}_m^\Phi(w) = \mathcal{B}(w) \cap \mathcal{R}_m^\Phi. \quad (16)$$

We denote $BS_i^\Phi(w)$ as the set of syndromes corresponding to the i decoding region $\mathcal{BR}_i^\Phi(w)$ of the decoder Φ , where $i \in \{c, f, m\}$ represents the correctable, failure, and miscorrectable region, respectively. $BS_i^\Phi(w)$ is defined as

$$BS_i^\Phi(w) = \{\mathbf{y} H^T \mid \mathbf{y} \in \mathcal{BR}_i^\Phi(w)\}. \quad (17)$$

The action space is defined as in pervious sections, i.e., $A = \{1, 2, \dots, n\}$. The reward function, similar to Equation (9), is defined as

$$R(s, a, s') = \begin{cases} -\frac{1}{L} + 1, & \text{if } s' \in \overline{BS}_f^\Phi(w) \cap S(w) \\ -\frac{1}{L}, & \text{if } s' \in BS_f^\Phi(w) \\ -\frac{1}{L} - 1, & \text{if } s' \in \overline{S}(w) \end{cases} \quad (18)$$

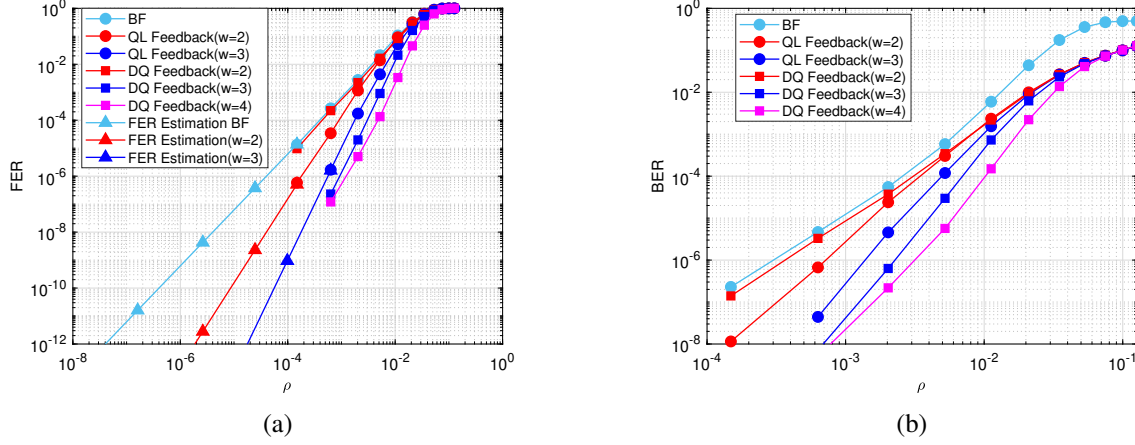


Fig. 2: Simulation results for the Tanner code using the Bit-Flipping decoder and its Q-learning and Deep-Q network feedback methods for truncated MDPs with radii $w = \{2, 3, 4\}$. (a) Frame Error Rate, (b) Bit Error Rate.

, where $\overline{S(w)}$ denotes the complement of the set $S(w)$. The error correction capability of bounded feedback decoder, after the convergence of training with the reward function defined in Equation (18) and based on Theorem 1 is given by

$$\min \left(\left\lfloor \frac{\min_{y \in \mathcal{BR}_f^\Phi(w)} w(y) + \min_{y \in \mathcal{BR}_m^\Phi(w)} w(y) - 1}{2} \right\rfloor, w \right) \quad (19)$$

where the minimum of the empty set is considered to be infinity, i.e., $\min \emptyset = \infty$. We can also define a reward function similar to Equation (11) as

$$R(s, a, s') = \begin{cases} -\frac{1}{L} + 1, & \text{if } s' = BS_c^\Phi(w) \\ -\frac{1}{L}, & \text{if } s' \in BS_f^\Phi(w) \\ -\frac{1}{L} - 1, & \text{if } s' \in BS_m^\Phi(w) \cup \overline{S(w)}. \end{cases} \quad (20)$$

The error correction capability of bounded feedback decoder, after the convergence of training with the reward function defined in Equation (20) and based on Remark 1, is given by

$$\min \left(\min_{y \in \mathcal{BR}_m^\Phi(w)} w(y) - 1, w \right). \quad (21)$$

V. NUMERICAL RESULTS

In this section, we present numerical results for the Tanner code with parameters (155, 64, 20). We select the arbitrary decoder Φ to be the bit-flipping decoder, which, in general, can be any well-established code and decoder pair from the literature. The goal is to enhance its performance through the integration of the RL block. The bounded error weight enumerator of the Tanner code under the BF decoder, around all-zero codeword, is $E(w = 4) = 10092825x^4 + 154225x^3 + 620x^2$, where w denotes the maximum weight of the error patterns considered during decoding. The multiplicity of x^i represents the number of uncorrectable weight- i errors under BF decoder. In addition, no miscorrections are observed for this code when employing the BF decoder for error patterns with weights of at most 4. In Figure 2, we illustrate the performance of the bit-flipping algorithm alongside truncated MDPs with radii $w = \{2, 3\}$,

trained using the Q-learning, and radii $w = \{2, 3, 4\}$, trained using the Deep Q-learning algorithms. We also present the frame error rate (FER) estimates in the error floor region for the feedback model learned using the Q-learning algorithm based on their uncorrectable error patterns. The results demonstrate that, instead of training the RL block to learn the entire code space, we can use the existing bit-flipping algorithm and incorporate the RL block as a feedback architecture to enhance its performance. The Q-learning algorithm with $w = \{2, 3\}$ achieves improved performance with guaranteed error correction. However, for $w = 4$, the state space becomes prohibitively large, making it impractical to use a Q-table. In this case, we train only the Deep Q-Network. Using the Deep-Q network, for $w = 2$, it does not effectively learn the decision boundary due to the small radius of the Hamming ball. However, for $w = 3$, as shown by comparison with the Q-table results, the network is able to generalize the optimal policy and successfully correct error patterns were not encountered during training. For $w = 4$, further performance improvement is observed in comparison to $w = 3$.

VI. CONCLUSIONS

In this paper, we propose a novel feedback method aimed at reducing the number of states in the MDP. Additionally, we focus on enhancing the performance of a chosen decoder by targeting only those error patterns that the decoder fails to correct. In essence, the feedback method seeks to efficiently expand the correctable region of the decoder. In general, our goal is to leverage the performance of existing well-designed decoders and refine their state space and using RL algorithms to further enhance their decoding performance.

REFERENCES

- [1] T. Richardson, "Error Floors of LDPC Codes," in *Proceedings of the Annual Allerton Conference on Communication Control and Computing*, vol. 41, no. 3. The University; 1998, October 2003, pp. 1426–1435.

- [2] B. Vasić, S. K. Chilappagari, D. V. Nguyen, and S. K. Planjery, "Trapping Set Ontology," in *2009 47th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, September 2009, pp. 1–7.
- [3] C. A. Kelley and D. Sridhara, "Pseudocodewords of Tanner Graphs," *IEEE Transactions on Information Theory*, vol. 53, no. 11, pp. 4013–4038, November 2007.
- [4] R. Koetter and P. O. Vontobel, "Graph-Covers and Iterative Decoding of Finite Length Codes," in *Proc. 3rd Intern. Symp. on Turbo Codes and Related Topics*, September 2003, pp. 1–5.
- [5] N. Varnica, M. P. C. Fossorier, and A. Kavcic, "Augmented Belief Propagation Decoding of Low-Density Parity Check Codes," *IEEE Transactions on Communications*, vol. 55, no. 7, pp. 1308–1317, July 2007.
- [6] T. Hehn, J. B. Huber, O. Milenkovic, and S. Laendner, "Multiple-Bases Belief-Propagation Decoding of High-Density Cyclic Codes," *IEEE Transactions on Communications*, vol. 58, no. 1, pp. 1–8, January 2010.
- [7] A. I. V. Casado, M. Griot, and R. D. Wesel, "LDPC Decoders with Informed Dynamic Scheduling," *IEEE Transactions on Communications*, vol. 58, no. 12, pp. 3470–3479, December 2010.
- [8] Y. Han and W. E. Ryan, "Low-Floor Decoders for LDPC Codes," *IEEE Transactions on Communications*, vol. 57, no. 6, pp. 1663–1673, June 2009.
- [9] S. K. Planjery, D. Declercq, L. Danjean, and B. Vasic, "Finite Alphabet Iterative Decoders—Part I: Decoding Beyond Belief Propagation on the Binary Symmetric Channel," *IEEE Transactions on Communications*, vol. 61, no. 10, pp. 4033–4045, October 2013.
- [10] D. Declercq, B. Vasic, S. K. Planjery, and E. Li, "Finite Alphabet Iterative Decoders—Part II: Towards Guaranteed Error Correction of LDPC Codes via Iterative Decoder Diversity," *IEEE Transactions on Communications*, vol. 61, no. 10, pp. 4046–4057, September 2013.
- [11] E. Nachmani, Y. Be'ery, and D. Burshtein, "Learning to Decode Linear Codes Using Deep Learning," in *2016 54th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, September 2016, pp. 341–346.
- [12] T. Gruber, S. Cammerer, J. Hoydis, and S. Ten Brink, "On Deep Learning-Based Channel Decoding," in *2017 51st Annual Conference on Information Sciences and Systems (CISS)*. IEEE, March 2017, pp. 1–6.
- [13] A. Bennatan, Y. Choukroun, and P. Kisilev, "Deep Learning for Decoding of Linear Codes - A Syndrome-Based Approach," in *2018 IEEE International Symposium on Information Theory (ISIT)*, June 2018, pp. 1595–1599.
- [14] L. Lugosch and W. J. Gross, "Neural Offset Min-Sum Decoding," in *2017 IEEE International Symposium on Information Theory (ISIT)*, June 2017, pp. 1361–1365.
- [15] S. Adiga, X. Xiao, R. Tandon, B. Vasić, and T. Bose, "Generalization Bounds for Neural Belief Propagation Decoders," *IEEE Transactions on Information Theory*, vol. 70, no. 6, pp. 4280–4296, February 2024.
- [16] F. Carpi, C. Häger, M. Martalò, R. Raheli, and H. D. Pfister, "Reinforcement Learning for Channel Coding: Learned Bit-Flipping Decoding," in *2019 57th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*. IEEE, September 2019, pp. 922–929.
- [17] M. Taghipour, A. K. Pradhan, and B. Vasić, "Reinforcement Learning Assisted Decoding," in *International Telemetry Conference 2024*, October 2024, pp. 1–5.
- [18] S. Habib, A. Beemer, and J. Klierer, "Belief Propagation Decoding of Short Graph-Based Channel Codes via Reinforcement Learning," *IEEE Journal on Selected Areas in Information Theory*, vol. 2, no. 2, pp. 627–640, April 2021.
- [19] W. Ryan and S. Lin, *Channel Codes: Classical and Modern*. Cambridge University Press, 2009.
- [20] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 2018.
- [21] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing Atari with Deep Reinforcement Learning," 2013. [Online]. Available: <https://arxiv.org/abs/1312.5602>
- [22] M. Taghipour and B. Vasic, "Action-List Reinforcement Learning Syndrome Decoding for Binary Linear Block Codes," July 2025. [Online]. Available: <https://arxiv.org/abs/2507.17893>